

NLP-Емо-01

Распознавание эмоций по тексту

Подробная инструкция для подготовки презентации

Этот документ содержит всю информацию, необходимую для создания презентации проекта. Для каждого слайда указано: заголовок, что написать, какие картинки вставить и что можно сказать при защите.

МГТУ им. Н.Э. Баумана, кафедра ИУ10

Группа ИУ10-44

Исполнители: Осипова П.С., Селина М.А.

Преподаватель: Буркацкий К.А.

Что это за проект (прочитай перед началом)

Мы написали программу на Python, которая определяет эмоцию в тексте. Пользователь вводит фразу на русском языке (например «Меня это бесит!»), а программа отвечает: «Эмоция: гнев, Уверенность: 92%».

Программа умеет распознавать 10 эмоций: радость, грусть, гнев, страх, удивление, отвращение, воодушевление, вина, стыд и нейтральность.

Мы сделали два подхода к решению задачи:

- 1) Простой (baseline): текст превращается в числа через TF-IDF, потом классифицируется LogisticRegression.
- 2) Продвинутый: используем нейросеть BERT (rubert-tiny2), которая понимает смысл текста. BERT работает лучше.

Программа работает через консоль (терминал). Есть команды /help и /exit. Все ошибки обрабатываются, программа не падает.

Датасет — ru-izard-emotions с Hugging Face, ~25 000 текстов на русском языке. Каждый текст размечен по 10 эмоциям (один текст может иметь несколько эмоций — это называется multi-label).

Словарь терминов (чтобы понимать о чём речь)

- **NLP** — Natural Language Processing — обработка естественного языка компьютером
- **TF-IDF** — способ превратить текст в числа, чтобы подать на вход модели. Считает, какие слова важны в конкретном тексте
- **BERT** — нейросеть-трансформер, которая «понимает» смысл слов в контексте. Мы используем rubert-tiny2 — версию для русского языка
- **Fine-tuning** — дообучение — берём готовую модель BERT и доучиваем её на наших данных об эмоциях
- **Лемматизация** — приведение слова к начальной форме: «бесит» → «бесить»
- **Стоп-слова** — слова, которые не несут смысла и удаляются: «и», «в», «на»
- **Accuracy** — доля правильных ответов (чем выше — тем лучше)
- **Precision** — из того, что модель назвала «гнев» — сколько реально гнев
- **Recall** — из всех текстов с гневом — сколько модель нашла
- **F1-score** — среднее между Precision и Recall (одна метрика вместо двух)
- **Confusion Matrix** — таблица-тепловая карта, показывающая какие эмоции модель путает между собой
- **Confidence** — уверенность модели в ответе (0–100%)
- **Multi-label** — один текст может иметь несколько эмоций одновременно
- **OneVsRest** — способ решения multi-label: обучаем отдельный классификатор для каждой эмоции

Детальное описание каждого слайда

Ниже для каждого слайда написано: какой заголовок поставить, что именно написать в теле слайда (можно копировать текст), и подсказки — что можно сказать голосом при защите.

СЛАЙД 1: ТИТУЛЬНЫЙ

Заголовок слайда:

«Распознавание эмоций по тексту»

Что написать на слайде:

- Шифр проекта: NLP-Емо-01
- МГТУ им. Н.Э. Баумана, кафедра ИУ10
- Группа: ИУ10-44
- Исполнители: Осипова П.С., Селина М.А.
- Преподаватель: Буркацкий К.А.
- 2025/2026 учебный год

Подсказка: Этот слайд стандартный. Оформи как титульный лист курсовой.

СЛАЙД 2: ПОСТАНОВКА ЗАДАЧИ

Заголовок слайда:

«Постановка задачи»

Что написать на слайде:

Цель: разработать программу для автоматического определения эмоциональной окраски текста на русском языке.

Принцип работы:

1. Пользователь вводит текст на русском языке
2. Программа выполняет предобработку и классификацию
3. Выводит метку эмоции и уверенность модели (%)

Пример:

Ввод: «Меня это бесит, я в ярости!»

Вывод: Эмоция: гнев, Уверенность: 92%

Подсказка: На защите скажи: «Задача — классификация текста по эмоциям. Пользователь вводит текст, программа говорит какая это эмоция и насколько она уверена.»

СЛАЙД 3: ОБЛАСТЬ ПРИМЕНЕНИЯ

Заголовок слайда:

«Область применения»

Что написать (3 пункта, каждый с иконкой или буллетом):

- Анализ тональности текстов в соцсетях и системах обратной связи
- Поддержка чат-ботов и систем мониторинга тональности
- Учебно-исследовательские цели (курсовые, дипломы по NLP)

Подсказка: На защите: «Программа может использоваться для автоматического анализа отзывов клиентов, мониторинга настроений в соцсетях, а также как учебный проект.»

СЛАЙД 4: ДАТАСЕТ

Заголовок слайда:

«Датасет: ru-izard-emotions»

Что написать (список фактов):

- Источник: Hugging Face (автор Djacon)
- Объём: ~25 000 текстов на русском языке
- 10 классов эмоций (модель Изарда)
- Разметка: multi-label — у текста может быть несколько эмоций
- Формат: CSV/JSON, кодировка UTF-8
- Разбиение: 80% обучение / 20% тест, seed=42

Список 10 эмоций (можно в виде таблицы на слайде):

Русский	English
Радость	joy
Грусть	sadness
Гнев	anger
Страх	fear
Удивление	surprise
Отвращение	disgust
Воодушевление	enthusiasm
Вина	guilt
Стыд	shame
Нейтральность	neutral

Подсказка: На защите: «Датасет содержит 25 тысяч текстов, размеченных по 10 эмоциям. Это multi-label задача — один текст может выражать и грусть, и страх одновременно.»

СЛАЙД 5: ПАЙПЛАЙН ОБРАБОТКИ

Заголовок слайда:

«Архитектура: пайплайн обработки текста»

Что нарисовать — схема из 5 блоков со стрелками:



Подсказка: Сделай красивую блок-схему из прямоугольников со стрелками. Каждый блок — этап обработки. Это ключевой слайд, он показывает как программа работает изнутри.

СЛАЙД 6: ПРЕДОБРАБОТКА ТЕКСТА

Заголовок слайда:

«Предобработка текста»

Что написать (4 этапа + пример):

1. Приведение к нижнему регистру (lowercase)
2. Удаление знаков препинания и спецсимволов
3. Удаление стоп-слов (NLTK, русский язык)
4. Лемматизация — слово → начальная форма (pymorphy2)

Пример (покажи на слайде как трансформируется текст):

Исходный:	«Меня это бесит, я в ярости!»
Lowercase:	«меня это бесит, я в ярости!»
Без пунктуации:	«меня это бесит я в ярости»
Без стоп-слов:	«бесит ярости»
Лемматизация:	«бесить ярость»

Подсказка: На защите: «Предобработка нужна, чтобы убрать шум из текста. Модели проще работать с чистыми леммами, чем с сырым текстом.»

СЛАЙД 7: МОДЕЛИ КЛАССИФИКАЦИИ

Заголовок слайда:

«Модели классификации»

Что написать — два блока:

Блок 1 — Baseline (простая модель):

- TF-IDF — превращает текст в числовой вектор
- LogisticRegression — классический алгоритм классификации
- OneVsRest — отдельный классификатор на каждую эмоцию
- Быстро обучается, но невысокая точность

Блок 2 — Основная модель (нейросеть):

- BERT (rubert-tiny2) — трансформер для русского языка
- Fine-tuning — дообучили предобученную модель на наших данных
- Понимает контекст слов (в отличие от TF-IDF)
- Значительно лучше по метрикам

Подсказка: На защите: «Мы реализовали два подхода. Baseline — классический TF-IDF + логистическая регрессия, и BERT — нейросеть-трансформер. BERT показал результаты значительно лучше, потому что он учитывает контекст слов в предложении.»

СЛАЙД 8: РЕЗУЛЬТАТЫ

Заголовок слайда:

«Результаты: сравнение моделей»

Что написать — таблица метрик:

Метрика	TF-IDF+LogReg	BERT (rubert-tiny2)
F1-score (macro)	0.29	0.49
Top-1 Accuracy	44.6%	47.1%
Recall (macro)	0.20	0.43

Также вставь картинку Confusion Matrix:

Файл: models/confusion_matrix_bert.png (или confusion_matrix_bert_single.png)

Это тепловая карта — показывает какие эмоции модель путает.

Вывод (напиши внизу слайда):

BERT превосходит baseline по всем метрикам. Трансформеры — более эффективный подход для классификации эмоций.

Подсказка: На защите объясни метрики: «F1-score — это баланс между точностью и полнотой. Accuracy — доля правильных ответов. BERT дал F1 = 0.49 против 0.29 у baseline — почти вдвое лучше.» Если спросят про Confusion Matrix — покажи, какие эмоции модель путает чаще всего.

СЛАЙД 9: ДЕМОНСТРАЦИЯ РАБОТЫ

Заголовок слайда:

«Демонстрация работы программы»

Что сделать:

Вставь скриншот работы программы в консоли. Чтобы сделать скриншот, запусти:

```
python3 main.py --bert
```

И введи несколько примеров:

Введите текст: Меня это бесит, я в ярости!

Эмоция: гнев

Уверенность: 92%

Введите текст: Какой прекрасный день сегодня!

Эмоция: радость

Уверенность: 78%

Введите текст: Мне так страшно...

Эмоция: страх

Уверенность: 85%

Подсказка: Если есть возможность — сделай живую демонстрацию на защите (запусти программу и введи текст прямо при комиссии). Если нет — хватит скриншота.

СЛАЙД 10: СТЕК ТЕХНОЛОГИЙ

Заголовок слайда:

«Стек технологий»

Что написать — таблица с пояснениями:

Не просто перечисли библиотеки — покажи ЗАЧЕМ каждая нужна. Сделай таблицу из трёх колонок:

Назначение	Технология	Зачем используется
Язык	Python 3.8+	Основной язык разработки
NLP	NLTK	Стоп-слова русского языка
NLP	pymorphy2	Лемматизация (морфоанализ)
NLP	re (regex)	Удаление пунктуации и спецсимволов
Векторизация	scikit-learn	TF-IDF: текст -> числовой вектор
Векторизация	Transformers	BERT-эмбединги (rubert-tiny2)
ML-модели	scikit-learn	LogReg, RandomForest, метрики
Нейросеть	PyTorch	Обучение и инференс BERT
Данные	pandas	Работа с таблицами данных
Данные	HF Datasets	Загрузка датасета
Графики	matplotlib	Confusion Matrix
Графики	seaborn	Тепловые карты
Сохранение	joblib / torch	Сериализация моделей (.pkl, .pt)

Подсказка: На защите: «Каждая технология в стеке решает конкретную задачу. Например, NLTK даёт нам стоп-слова русского языка, pymorphy2 приводит слова к начальной форме, а scikit-learn — это и векторизация через TF-IDF, и baseline-модели, и расчёт метрик.»

СЛАЙД 11: ВЫВОДЫ

Заголовок слайда:

«Выводы»

Что написать (3-4 пункта):

1. Разработана программа классификации эмоций в тексте на русском языке
2. Реализованы два подхода: TF-IDF + LogReg (baseline) и BERT (rubert-tiny2)
3. BERT показал F1-score 0.49 — почти вдвое лучше baseline (0.29)
4. Все требования ТЗ выполнены: предобработка, векторизация, классификация, оценка качества, консольный интерфейс

Подсказка: На защите: «В результате работы мы убедились, что трансформеры значительно превосходят классические методы на задаче классификации эмоций. Программа полностью соответствует ТЗ и готова к эксплуатации.»

Структура проекта (для справки)

```

NLP_Емо/
├─ main.py           # Точка входа (запуск CLI)
├─ train.py          # Обучение TF-IDF модели
├─ train_bert.py     # Обучение BERT (multi-label)
├─ train_bert_single.py # Обучение BERT (single-label)
├─ src/
│   ├─ data_loader.py # Загрузка датасета (HF / CSV / JSON)
│   ├─ preprocessing.py # Предобработка текста
│   ├─ vectorizer.py   # TF-IDF векторизация
│   ├─ model.py        # Классическая ML-модель
│   ├─ bert_model.py   # BERT fine-tuning и инференс
│   ├─ evaluate.py     # Метрики и Confusion Matrix
│   └─ cli.py          # Консольный интерфейс
├─ models/           # Сохранённые модели и графики
├─ requirements.txt   # Зависимости Python
└─ README.md         # Документация

```

Как запустить программу (если нужен скриншот)

Установка зависимостей

```
pip install -r requirements.txt
```

Обучение модели

```

python3 train.py           # TF-IDF + LogReg (быстро)
python3 train_bert.py      # BERT multi-label (дольше)
python3 train_bert_single.py # BERT single-label

```

Запуск программы

```

python3 main.py           # TF-IDF модель (по умолчанию)
python3 main.py --bert    # BERT модель

```

Файлы для вставки в презентацию

Готовые картинки Confusion Matrix (тепловые карты):

- models/confusion_matrix.png — для TF-IDF модели
- models/confusion_matrix_bert.png — для BERT multi-label

- `models/confusion_matrix_bert_single.png` — для BERT single-label

Вставь одну из этих картинок на слайд 8 (Результаты). Лучше всего подойдёт `confusion_matrix_bert_single.png` — она проще для понимания.